

工业中的有限元方法

——作业一——

一、 作业任务

对给定的 $\left[x_0, x_{\frac{1}{2}}, x_1, \dots, x_n, x_{n+\frac{1}{2}}, x_n\right]$ 及 $\left[u_0, u_{\frac{1}{2}}, u_1, \dots, u_n, u_{n+\frac{1}{2}}, u_n\right]$, 利用二次基函数 $\varphi_i(x)$, 求任意一点 $x \in [a, b]$ 的函数值 u

二、 基本原理

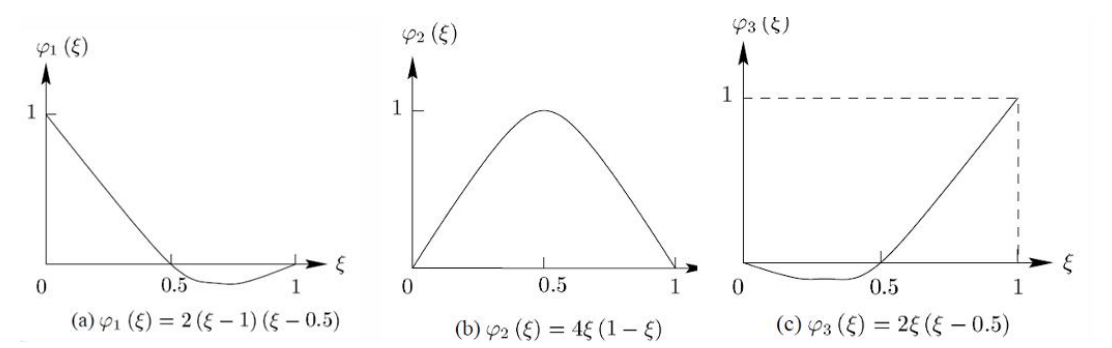


图 1 二次基函数

通过利用归一化，将任意区间的数值归一化至 $[0,1]$ 区间，则通过 $[0,1]$ 区间上的标准二次基函数进行数值计算。

三、 实验结果

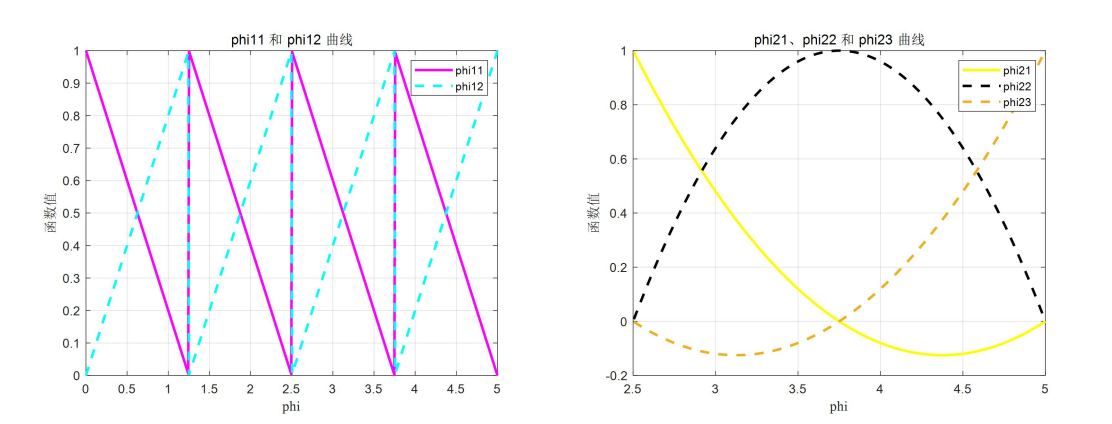


图 2 基函数求解实际展示图

为了能够更加直观感受基函数如何进行有限元分析，因此，分别对一次基函

数与二次基函数进行计算并进行绘制，如图 2 所示。与此同时，为了能够直观感受到参数 n 对于有限元求解的影响，并探究一次基函数与二次基函数的逼近差距，本作业对其进行了比较，如图 3 所示。

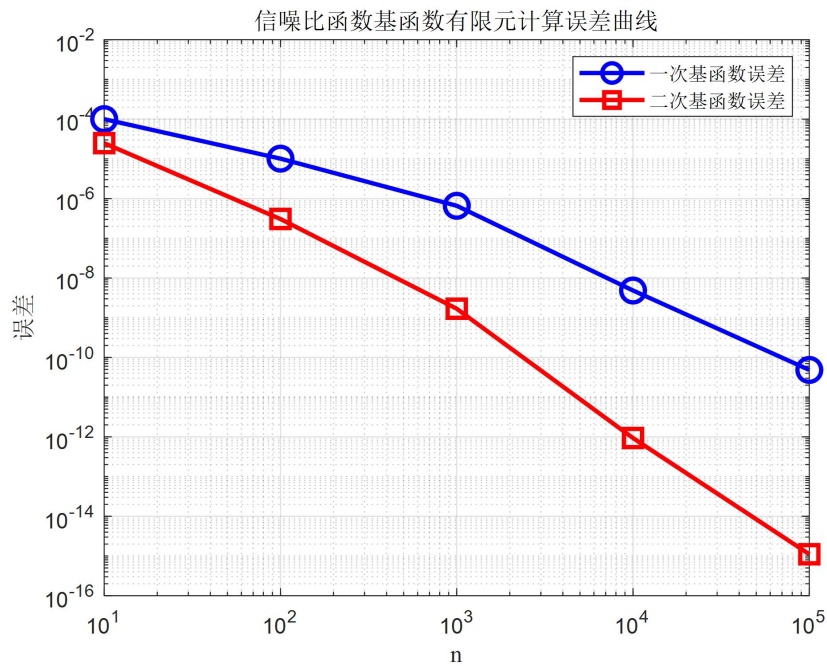


图 3 基函数求解实际展示图

具体而言，由于本人专业维电子信息，所以采用对数公式对单个点的函数值进行模拟，即：

$$u = \ln(1 + x)$$

如图 3 所示，单独研究参数 n 可以发现，由于给定点的不断增多，有限元逼近的数值方法误差逐渐降低。横向对比，当在同一 n 值下，很明显二次基函数的误差远低于一次基函数，大大提升了精度。

除此之外，本作业也对 $[0,5]$ 上的对数公式进行 50 点的数值计算，此时采用参数 $n=5$ ，对应产生的每条曲线如图 4 所示。

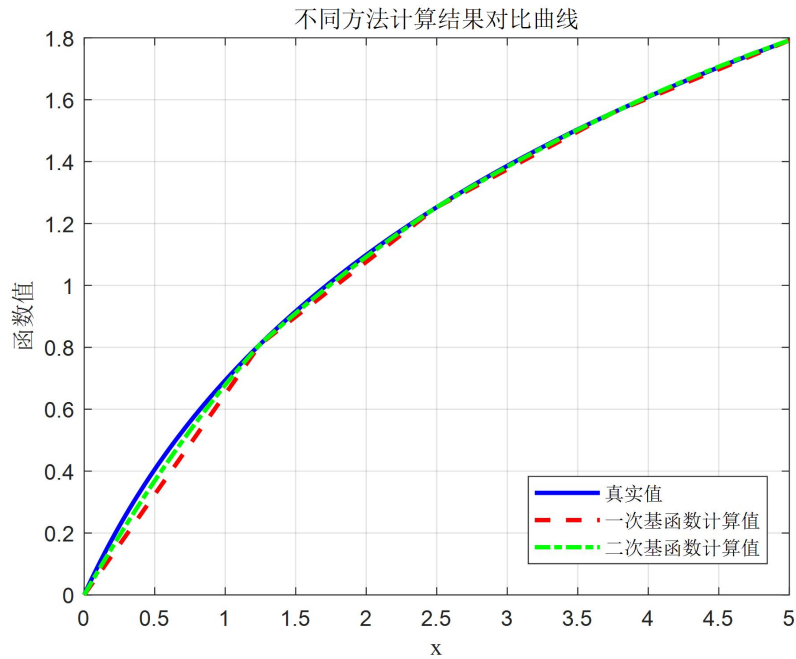


图 4 不同计算方法对比曲线

如图所示，同样可以证明，二次基函数相较于一次基函数的精度更加准确。

四、 实验代码

函数 Integral_linear_rep: 实现二次基函数表示

```
function [u,phi1,phi2,phi3] = Integral_liner_rep(n,xn,un,x)
% 假设 xn(1) <= x <= xn(n)
k = max(find(x >= xn)); % 寻找到相关子区间
if(k == n) k = k - 2; end
if(k == n - 1) k = k - 1; end % 处理边界条件
Xevion = (x - xn(k)) / (xn(k + 2) - xn(k)); % 归一化到 [0,1]
phi1 = 2 * (Xevion - 1) * (Xevion - 0.5); % 二次基函数 phi_1
phi2 = 4 * Xevion * (1 - Xevion); % 二次基函数 phi_2
phi3 = 2 * Xevion * (Xevion - 0.5); % 二次基函数 phi_3
u = un(k) * phi1 + un(k + 1) * phi2 + un(k + 2) * phi3;
end
```

主函数:

```
clc;
clear;

%% 测试单个点误差 比较一次基函数和二次基函数

x = 1.113;
n = 10;
error1 = [];
error2 = [];
n_values = [];

threshold = 1E5;

while n <= threshold
    xn = linspace(0, 5, n);
    un = log(xn + 1);

    % 这里假设linearrep和Integral_liner_rep函数已经定义
    [u1,f1,f2] = linearrep(n,xn,un,x);
    [u2,f3,f4] = Integral_liner_rep(n,xn,un,x);

    u_true = log(x + 1);

    error1 = [error1; abs(u_true - u1)];
    error2 = [error2; abs(u_true - u2)];
    n_values = [n_values; n];

    n = n * 10;
end

% 绘制误差曲线, 使用loglog函数让横轴和纵轴都为对数坐标, 并加粗曲线
figure;
loglog(n_values, error1, 'b-o', 'DisplayName', '一次基函数误差', 'LineWidth', 2, 'MarkerSize', 11);
loglog(n_values, error2, 'r-s', 'DisplayName', '二次基函数误差', 'LineWidth', 2, 'MarkerSize', 11);

% 添加标题和标签
title('信噪比函数基函数有限元计算误差曲线');
xlabel('n', 'FontName', 'Times New Roman');
ylabel('误差');

% 添加图例
legend;

% 网格线
grid on;

%% 绘制对应区间的曲线 比较误差
Sample_num = 512;

x = linspace(0, 5, Sample_num);
u_true = log(x + 1);
u1 = zeros(1, Sample_num);
u2 = zeros(1, Sample_num);
phi11 = zeros(1, Sample_num);
phi12 = zeros(1, Sample_num);
phi21 = zeros(1, Sample_num);
phi22 = zeros(1, Sample_num);
phi23 = zeros(1, Sample_num);
```

```

n = 5; % 可调参数
xn = linspace(0, 5, n);
un = log(xn + 1);

for i = 1:length(xn)
    [u1(i), phi11(i), phi12(i)] = linearrep(n, xn, un, x(i));
    [u2(i), phi21(i), phi22(i), phi23(i)] = Integral_liner_rep(n, xn, un, x(i));
end

% 绘制三条曲线
figure;
plot(xn, u_true, 'b-', 'DisplayName', '真实值', 'LineWidth', 2, 'MarkerSize', 11);
hold on;
plot(xn, u1, 'r--', 'DisplayName', '一次基函数计算值', 'LineWidth', 2, 'MarkerSize', 11);
plot(xn, u2, 'g-.', 'DisplayName', '二次基函数计算值', 'LineWidth', 2, 'MarkerSize', 11);
hold off;

% 添加标题和标签
title('不同方法计算结果对比曲线');
xlabel('x', 'FontName', 'Times New Roman');
ylabel('函数值');

% 添加图例
legend;

% 显示网格线
grid on;

% 绘制 phi11 和 phi12 曲线
figure;
plot(xn, phi11, 'm-', 'DisplayName', 'phi11', 'LineWidth', 2, 'MarkerSize', 11);
hold on;
plot(xn, phi12, 'c--', 'DisplayName', 'phi12', 'LineWidth', 2, 'MarkerSize', 11);
hold off;
title('phi11 和 phi12 曲线');
xlabel('phi', 'FontName', 'Times New Roman');
ylabel('函数值');
legend;
grid on;

% 绘制 phi21、phi22 和 phi23 曲线
figure;
plot(xn, phi21, 'y-', 'DisplayName', 'phi21', 'LineWidth', 2, 'MarkerSize', 11);
plot(xn, phi22, 'k--', 'DisplayName', 'phi22', 'LineWidth', 2, 'MarkerSize', 11);
plot(xn, phi23, 'b-.', 'DisplayName', 'phi23', 'LineWidth', 2, 'MarkerSize', 11);
hold off;
title('phi21、phi22 和 phi23 曲线');
xlabel('phi', 'FontName', 'Times New Roman');
ylabel('函数值');
legend;
grid on;

```